



SSC502 – Laboratório de ICC I

Alocação Dinâmica

Prof.Dr. Danilo Spatti

- Sempre que **escrevemos** um programa, é preciso **reservar espaço** para as **informações** que serão **processadas**.
- Uma variável é uma **posição** de **memória** que **armazena** uma informação **que pode** ser **modificada** pelo programa.
- Ela deve ser **definida antes** de ser usada.

- Nem sempre é possível saber, em **tempo de execução**, o quanto de **memória** um **programa** irá **precisar**.
- Exemplo: Faça um programa para cadastrar o preço de N produtos, em que N é um valor informado pelo usuário.

```
int N, i;  
double produtos[N];
```

Errado! Não sabemos o valor de N

```
int N, i;  
  
scanf ("%d", &N)  
  
double produtos[N];
```

Funciona, mas não é o mais indicado

- A alocação **dinâmica** permite ao programador **criar** “variáveis” em **tempo** de **execução**, ou seja, alocar **memória** para novas **variáveis** quando o **programa** está sendo **executado**, e não apenas quando se está **escrevendo** o programa.
- Quantidade de **memória** é **alocada** sob **demanda**, ou seja, quando o programa **precisa**.

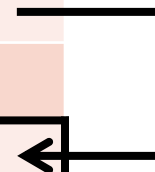
- Menos desperdício de memória.
- Espaço é reservado até liberação explícita.
- Depois de liberado, estará disponibilizado para outros usos e não pode mais ser acessado.
- Espaço alocado e não liberado explicitamente é automaticamente liberado ao final da execução.

Memória		
posição	variável	conteúdo
119		
120		
121	int *p	NULL
122		
123		
124		
125		
126		
127		
128		

Alocando 5
posições de
memória em int *p



Memória		
posição	variável	conteúdo
119		
120		
121	int *p	123
122		
123	p[0]	11
124	p[1]	25
125	p[2]	32
126	p[3]	44
127	p[4]	52
128		



- A linguagem **C ANSI** usa **apenas** 4 funções para o **sistema** de **alocação** dinâmica, disponíveis na **stdlib.h**:

malloc

calloc

realloc

free

```
void *malloc (unsigned int num);
```

- Dado o número de **bytes** que queremos alocar (**num**), ela **aloca** na memória e **retorna** um **ponteiro** `void*` para o **primeiro** byte **alocado**.


```
void *malloc (unsigned int num) ;
```

- O ponteiro `void*` pode ser **atribuído** a **qualquer tipo de ponteiro** via **type cast**.
- Se **não houver** memória **suficiente** para **alocar** a memória **requisitada**, a função `malloc()` retorna um **ponteiro nulo**.

- Alocar 1000 bytes de memória livre:

```
char *p;
```

```
p = (char *) malloc(1000) ;
```

- Alocar espaço para 50 inteiros:

```
int *p;
```

```
p = (int *) malloc(50*sizeof(int))
```

```
1  # include <stdio.h>
2  # include <stdlib.h>
3
4  int main()
5  {
6      int *p, i;
7      p = (int *) malloc(5*sizeof(int));
8      if (p == NULL)
9      {
10         printf("Memoria Insuficiente!\n");
11         system("pause");
12         exit(1);
13     }
14     for (i = 0; i < 5; i++)
15     {
16         printf("Digite o valor da posicao %d :", i);
17         scanf("%d", &p[i]);
18     }
19     return 0;
20 }
```

- Basicamente, a função `calloc()` faz o **mesmo** que a função `malloc()`. A diferença é que agora **passamos** a **quantidade** de **posições** a serem **alocadas** e o **tamanho** do tipo de **dado** alocado como **parâmetros** distintos da função.

```
void *calloc(unsigned int num, unsigned int size);
```

```
1  # include <stdio.h>
2  # include <stdlib.h>
3
4  int main()
5  {
6      int *p, *p1;
7      p = (int *) malloc(50*sizeof(int));
8      if (p == NULL) printf("Memoria Insuficiente!\n");
9      p1 = (int *) calloc(50,sizeof(int));
10     if (p1 == NULL) printf("Memoria Insuficiente!\n");
11     return 0;
12 }
```

- A função `realloc` modifica o tamanho da memória previamente `alocada` e apontada por `*ptr` para aquele especificado por `num`.
- O valor de `num` pode ser maior ou menor que o original.
- A função `realloc()` serve para `realocar memória` e tem o seguinte protótipo:

```
void *realloc(void *ptr, unsigned int num);
```

- Um ponteiro para o **bloco** é **devolvido** porque `realloc()` **pode** precisar **mover** o **bloco** para aumentar seu **tamanho**.
- Se isso **ocorrer**, o **conteúdo** do bloco antigo é **copiado** para o **novo bloco**, e nenhuma **informação** é perdida.

```
void *realloc(void *ptr, unsigned int num);
```

- Se `*ptr` for **nulo**, aloca `num` bytes e devolve um **ponteiro** (igual `malloc`).
- Se `num` é **zero**, a memória **apontada** por `*ptr` é **liberada** (igual `free`).
- Se **não** houver **memória suficiente** para a **alocação**, um ponteiro **nulo** é **devolvido** e o **bloco** original é deixado **inalterado**.


```
1  # include <stdio.h>
2  # include <stdlib.h>
3
4  int main()
5  = {
6      int i;
7      int *p = malloc(5*sizeof(int));
8      for (i = 0; i < 5; i++) p[i] = i+1;
9      for (i = 0; i < 5; i++) printf("%d\n",p[i]);
10     printf("\n");
11     p = realloc(p,3*sizeof(int));
12     for (i = 0; i < 3; i++) printf("%d\n",p[i]);
13     printf("\n");
14     p = realloc(p,10*sizeof(int));
15     for (i = 0; i < 10; i++) printf("%d\n",p[i]);
16     return 0;
17 }
```

```
1
2
3
4
5
1
2
3
4
5
6649957
-1241513802
22645
12334456
12324168
Process returned 0 (0x0)   execution time : 0.016 s
```

```
void free(void *p) ;
```

- Diferente das **variáveis definidas** durante a **escrita** do programa, as **variáveis** alocadas **dinamicamente não são** liberadas **automaticamente** pelo programa.
- Quando alocamos **memória dinamicamente** é **necessário** que nós a **liberemos** quando ela **não** for mais **necessária**.

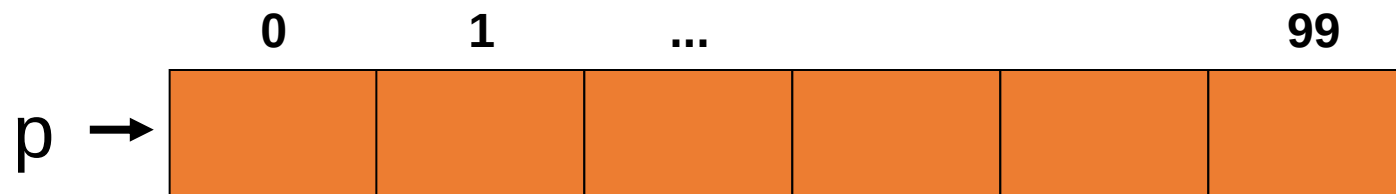
```
void free(void *p);
```

- Assim, para **liberar** a **memória**, basta passar como **parâmetro** para a função `free()` o **ponteiro** que aponta para o **início** da **memória** a ser **liberada**.
- Quando se **aloca** a **memória**, o programa **guarda** o **número** de bytes alocados numa "tabela de alocação" **interna**.

```
1  # include <stdio.h>
2  # include <stdlib.h>
3
4  int main()
5  {
6      int *p, i;
7      p = (int *) malloc(50*sizeof(int));
8      if (p == NULL)
9      {
10         printf("Memoria Insuficiente!\n");
11         system("pause");
12         exit(1);
13     }
14     for (i = 0; i < 50; i++) p[i] = i+1;
15     for (i = 0; i < 50; i++) printf("%d\n",p[i]);
16     free(p);
17     return 0;
18 }
```

- Ao alocar memória, na verdade está se alocando vetores.

```
int *p;  
int i, N = 100;  
  
p = (int *) malloc(N*sizeof(int));  
  
for (i = 0; i < N; i++)  
    scanf("%d", &p[i]);
```



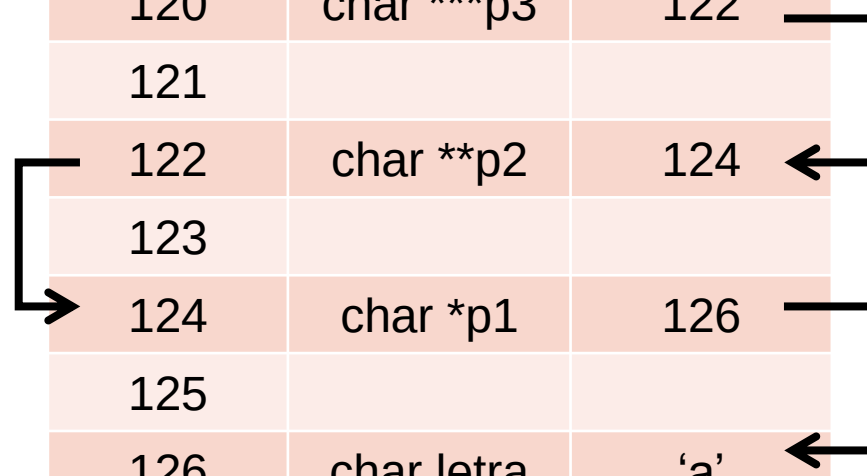
- Para liberá-lo da memória, basta chamar a função `free()` ao final do programa:

```
int *p;  
int i, N = 100;  
  
p = (int *) malloc(N*sizeof(int));  
  
for (i = 0; i < N; i++)  
    scanf("%d", &p[i]);  
  
free(p);
```

- Para alocar matrizes e vetores multidimensionais, utiliza-se o conceito de **ponteiro para ponteiro**.

```
char letra = 'a';  
char *p1;  
char **p2;  
char ***p3;  
  
p1 = &letra;  
p2 = &p1;  
p3 = &p2;
```

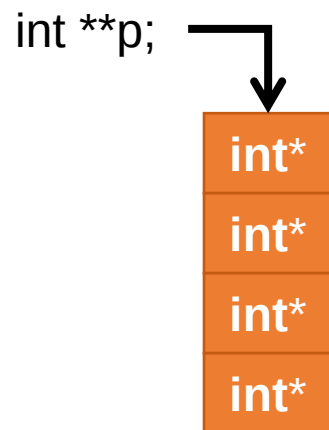
Memória		
posição	variável	conteúdo
119		
120	char ***p3	122
121		
122	char **p2	124
123		
124	char *p1	126
125		
126	char letra	'a'
127		



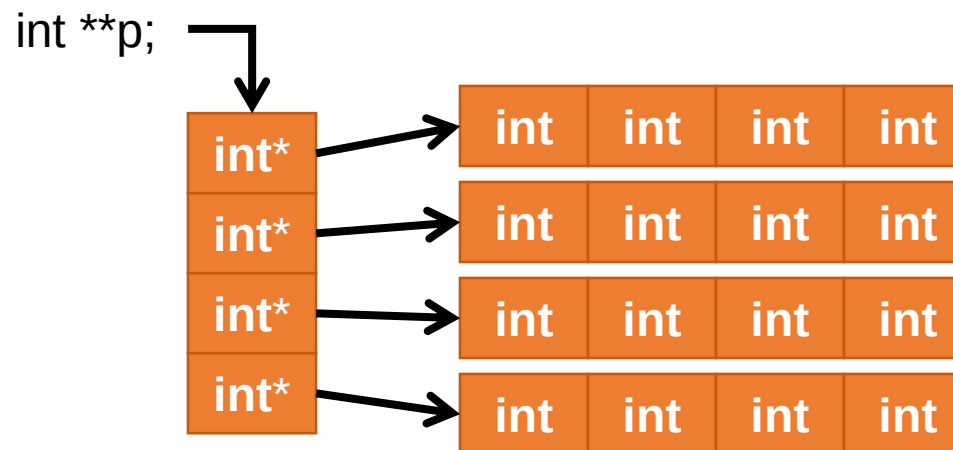
- Em um **ponteiro para ponteiro**, cada **nível** do **ponteiro** permite **criar** uma nova **dimensão** no vetor.

```
p = (int**) malloc(N*sizeof(int*));  
  
for (i = 0; i < N; i++){  
    p[i] = (int *)malloc(N*sizeof(int));  
}
```

1º malloc:
cria as linhas



2º malloc:
cria as colunas



- Para liberar um vetor com **mais** de uma **dimensão** da **memória**, é **preciso liberar** a memória **alocada** em **cada** uma de suas **dimensões**, na ordem **inversa** da que foi alocada.

```
int **p; //2 "*" = 2 níveis = 2 dimensões
int i, j, N = 2;
p = (int**) malloc(N*sizeof(int*));

for (i = 0; i < N; i++){
    p[i] = (int *)malloc(N*sizeof(int));
    for (j = 0; j < N; j++)
        scanf("%d", &p[i][j]);
}
```

```
for (i = 0; i < N; i++)
    free(p[i]);
free(p);
```

- As regras são **exatamente** as **mesmas** para a alocação de uma **struct**.
- Para alocar uma **única struct**, um **ponteiro** para struct receberá o `malloc()`.
- Utilizamos o **operador []** para **acessar** o conteúdo.
- Usamos `free()` para **liberar** a memória **alocada**.

```
struct cadastro{
    char nome[50];
    int idade;
};

int main(){
    struct cadastro *vcad = (struct cadastro*) malloc(10*sizeof(struct cadastro));

    strcpy(vcad[0].nome, "Maria");
    vcad[0].idade = 30;

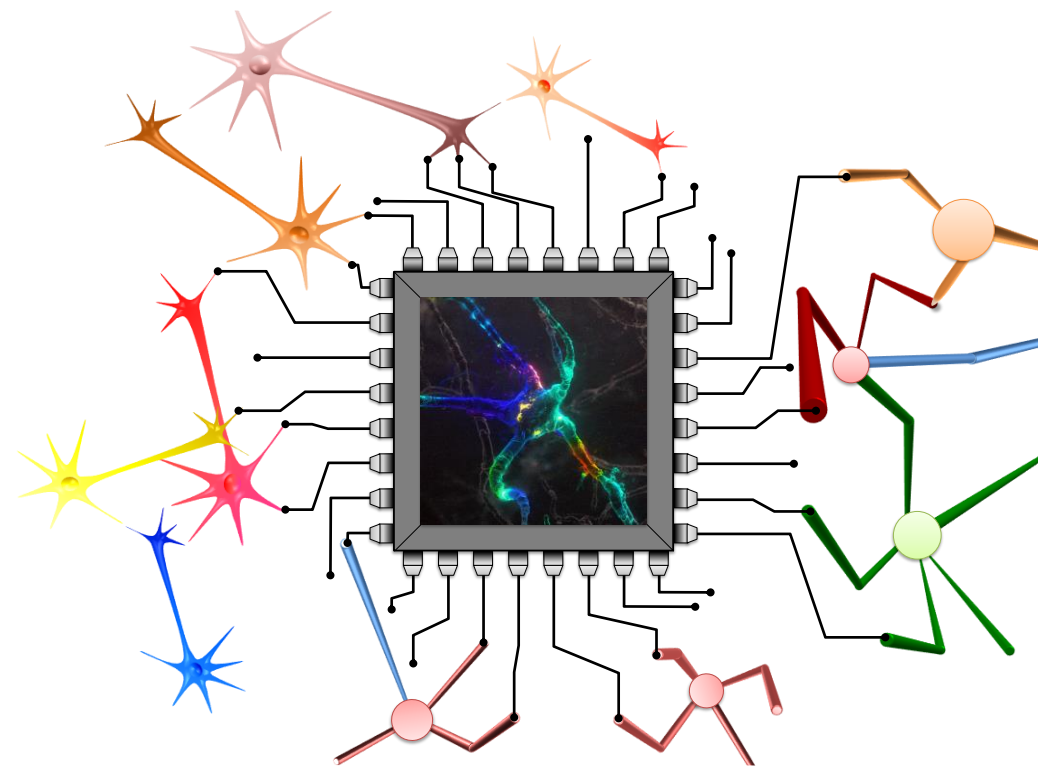
    strcpy(vcad[1].nome, "Cecilia");
    vcad[1].idade = 10;

    strcpy(vcad[2].nome, "Ana");
    vcad[2].idade = 10;

    free(vcad);

    return 0;
}
```

spatti@icmc.usp.br



GE4Bio – Grupo de Estudos em Sinais Biológicos